

NWSDB Service Oriented Computing Solution

CSE5013 Service Oriented Computing WRIT1

Student Name: [Replace with your name]
Student ID: [Replace with your student ID]
Submission date: [Replace before submission]

Acknowledgement

I would like to express my sincere gratitude to my module leader and lecturer for Service Oriented Computing CSE5013 for their guidance, feedback and direction throughout this assignment. I am also grateful to ICBT Campus for the academic resources and learning environment that supported this work.

I acknowledge the documentation and open-source communities behind ASP NET Core, Entity Framework Core, React, Docker and Kubernetes. Their tools and documentation supported the implementation and evaluation of the NWSDB service-oriented solution.

Contents

1. Executive Summary
2. Introduction
3. Task 1 Architecture Comparison and SOA Justification
4. Task 2 SOC Application Requirements Design and Development
5. Task 2 Code Quality Reusability and Maintainability
6. Task 3 Testing and Debugging
7. Task 4 Deployment Techniques
8. Overall Conclusion

References

Appendix A Screenshot Evidence Checklist

1. Executive Summary

This report presents a service-oriented computing solution for the Sri Lanka National Water Supply and Drainage Board. The solution addresses the requirement to expose water-usage information and bill-payment facilities to the NWSDB website and future third-party applications. The implemented system separates identity, payment and usage responsibilities into independently deployable ASP NET Core Web APIs, with a React client consuming their REST interfaces. JWT authentication and role-based authorisation protect customer and operational functions.

The project demonstrates the core advantages expected from service-oriented architecture: business capabilities are separated behind explicit service contracts, the client is not tied to database details, and individual services can be changed or deployed independently. The strongest evidence is the working customer portal, protected payment and meter-reading APIs, Swagger documentation, Docker definitions, Kubernetes manifests, automated tests and the PayHere sandbox checkout. The current repository is suitable as an academic demonstration. Before a production deployment, the InMemory data stores, development JWT fallback, open CORS policy and incomplete Kubernetes ingress routing must be replaced or hardened.

2. Introduction

NWSDB intends to improve customer service while reducing operational cost. Customers need timely information about their water usage and bills, and they need convenient ways to pay. The same capabilities must be available to the NWSDB website and to third-party partners across organisational boundaries. This makes a service-based design more appropriate than a user-interface-only application because reusable, documented APIs become first-class deliverables.

The assignment requires an explanation and comparison of monolithic and service-oriented architectures, a designed and developed SOC application with client consumption and diagrams, testing and debugging evidence, and an explanation of suitable deployment techniques. The following report maps the actual codebase to those four assessment areas. It distinguishes implemented evidence from the remaining production improvements so that the submission does not overstate what has been deployed.

Task 1 Architecture Comparison and SOA Justification

1.1 Monolithic Architecture

A monolithic design would place authentication, usage calculation, meter-reading entry, payment creation, payment-gateway integration and the web interface in one deployable application and usually one shared data model. It is initially straightforward to develop, test and deploy because the code runs in a single process. Internal method calls are simple and transactions can be local. However, a change to payment processing would require redeploying the same unit that supplies meter information and login. Growth in one capability also requires scaling the whole application, which wastes resources when demand is uneven.

1.2 Service Oriented Architecture

Service-oriented architecture organises software around autonomous business capabilities exposed through contracts. In this solution the Identity Service issues user tokens; the Usage Service records readings and calculates an estimated tiered bill; and the Payment Service records payments and constructs a PayHere checkout request. Their boundaries align with business ownership and each service has its own DbContext. The React application is a consumer of HTTP JSON APIs rather than a direct consumer of service data. This reflects the REST approach of resource-oriented interfaces over HTTP described by Fielding (2000) and the loose-coupling principles described by Erl (2005).

1.3 Comparison and Trade Offs

The comparison is not absolute. SOA introduces network calls, distributed authentication, configuration management and eventual consistency concerns. A monolith can be a sensible early option for a small team. For NWSDB, however, the integration requirement and the different change rates of payment, meter usage and identity make the SOA trade-off justified. Payment partners can integrate against the Payment Service without obtaining access to meter data. The Usage Service can be enhanced with new tariff rules without redesigning the payment API. This is more maintainable because boundaries limit the effect of change, and more scalable because a busy service can later be replicated independently once durable shared storage is introduced.

1.4 Maintainability

Maintainability is therefore considered at both code and operational levels. In a monolith, a defect or release in one feature creates a release event for every feature. Dependencies may be hidden inside the same codebase, and teams can accidentally share models across unrelated business concerns. In the proposed SOA, a published route, request model and response model are the explicit agreement between provider and consumer. This agreement permits versioning and documentation through Swagger. The separation is not simply a folder structure: each service has its own executable project, middleware pipeline, health endpoint, Dockerfile and data context. These details make independent deployment possible when a future organisation or partner needs a specific capability.

1.5 Scalability and Justification

Scalability should be understood as the ability to respond to workload without unnecessary duplication. A billing-period spike may increase calls to the Usage Service, while a payment deadline may increase checkout requests. In a monolith, adding instances duplicates every function even when only one is under pressure. With a durable database and stateless service instances, the selected architecture can increase replicas for the affected API while keeping the other APIs stable. The present prototype deliberately keeps one replica because each InMemory database is local to a process. This limitation is recognised in the deployment section rather than being hidden.

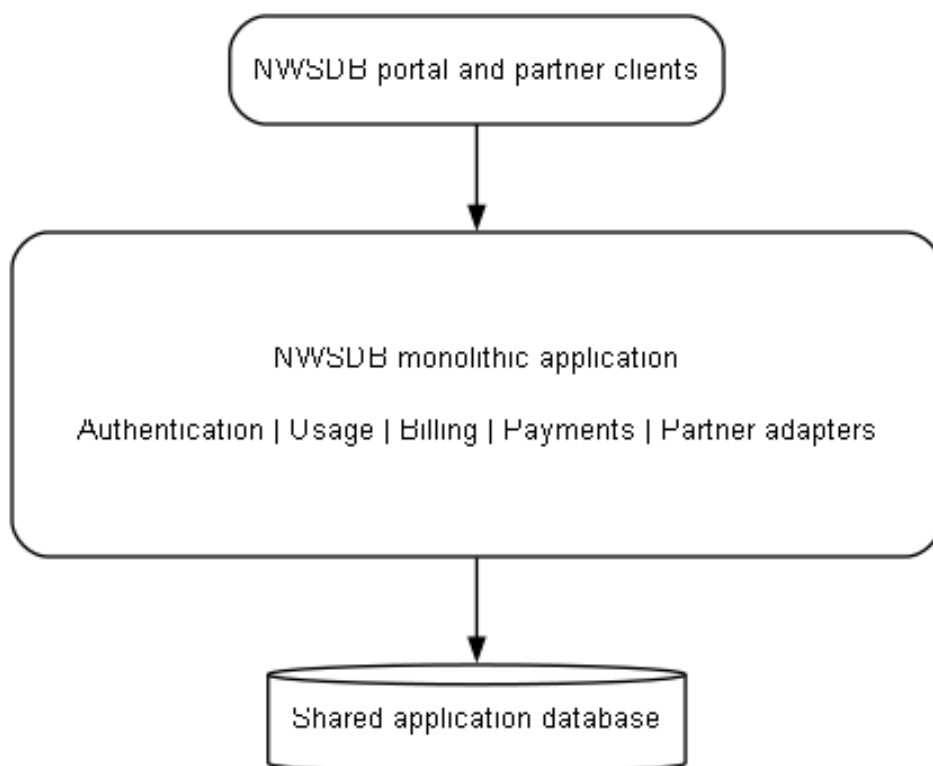


Figure 1 Monolithic alternative for NWSDB

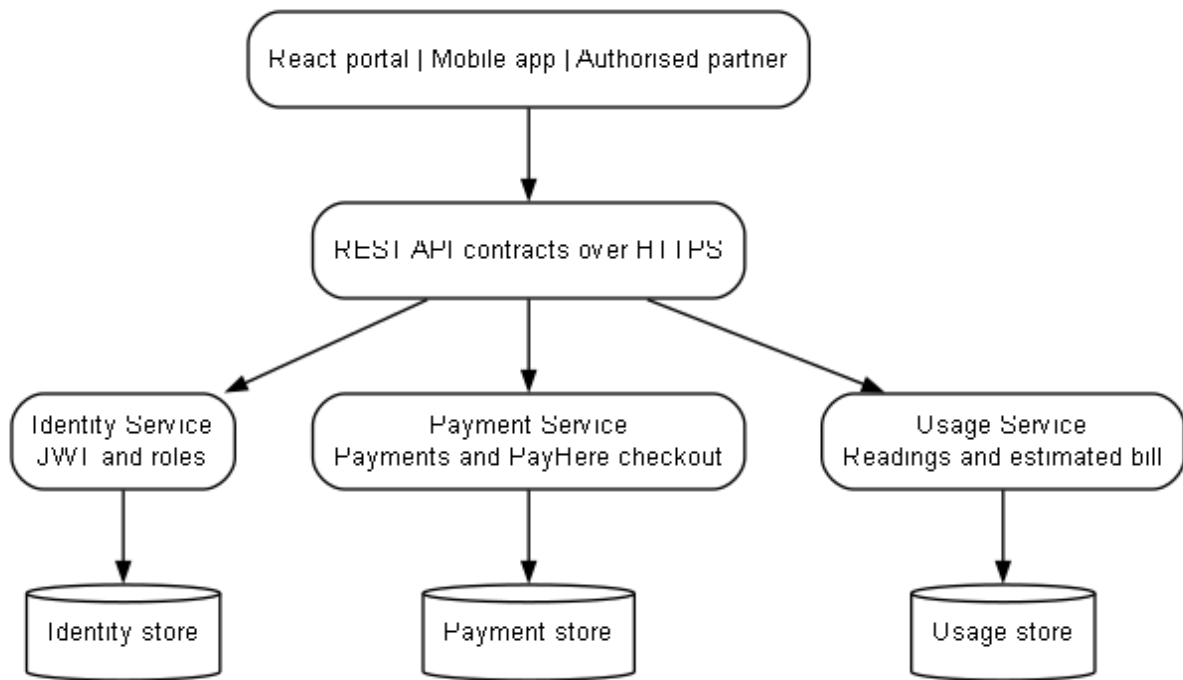


Figure 2 Selected service oriented architecture

Task 2 SOC Application Requirements Design and Development

2.1 System Requirements and Services

The implementation contains three ASP NET Core services and one React client. Identity Service exposes registration, login, current-user and user-administration endpoints. It stores password hashes, issues JWTs containing role and account-number claims, and distinguishes Customer, Staff and Admin roles. Payment Service exposes account payment history, payment creation, status updating and PayHere checkout/notification endpoints. It verifies that a customer can access only the account contained in the token, while staff and administrators can perform authorised operational functions. Usage Service stores meter readings, blocks a new reading below the prior reading, returns account history and calculates a progressive demonstration tariff.

2.2 API Architecture

Controllers are thin HTTP adapters and delegate business operations to service classes through interfaces such as `IPaymentService` and `IUsageService`. This separates routing, status-code handling and access checks from the main business logic. The use of `ControllerBase`, attribute routing, response status metadata and Swagger/OpenAPI follows the controller-based ASP NET Core Web API model described by Microsoft (2026). DTO-style request and response records make the API contract clearer than exposing internal entities directly in every operation.

2.3 Client and PayHere Integration

The React client centralises API calls in `nwsdbApi.js` and attaches the bearer token when needed. Its payment panel supports a direct portal payment and a PayHere sandbox flow. For the PayHere flow, the server constructs the form fields and calculates the hash; the merchant secret never enters the browser. The notification endpoint validates the returned signature before changing a pending payment status. This is a suitable example of a third-party service interaction, although PayHere requires the configured Integration domain and merchant secret to match the browser origin (PayHere, 2026).

2.4 Design Diagram Set

The diagram set is deliberately varied. The monolithic and SOA diagrams make the architectural comparison visible. The system architecture diagram shows runtime communication; use-case and activity diagrams explain the main actors and payment flow; sequence diagrams identify API exchanges; the class and entity relationship diagrams show

implementation structure; and the deployment diagram connects the design to Docker and Kubernetes.

2.5 REST API Contract

The service contracts use conventional HTTP semantics. GET operations obtain a resource or resource collection, POST creates a payment, user or meter reading, and PATCH changes a payment status. Controllers return useful status outcomes such as 201 Created for successful creation, 400 Bad Request for invalid inputs, 401 Unauthorized for failed login, 403 Forbidden for role or account violations and 404 Not Found for missing records. This makes the APIs understandable to the React client and to future partner developers. Health endpoints provide a lightweight operational contract for local diagnosis, Docker health checks and Kubernetes probes.

2.6 Usage and Billing Logic

The current calculation is intentionally labelled an estimated demonstration tariff. It applies progressive price tiers to the consumption difference between the newest and previous readings. This is appropriate for demonstrating service logic and testable decision branches, but production tariff rules would be sourced from an approved NWSDB policy table and may need dates, tariffs, taxes, customer categories, arrears and audit controls. Keeping the rule inside the Usage Service makes this future change local to the responsible business capability.

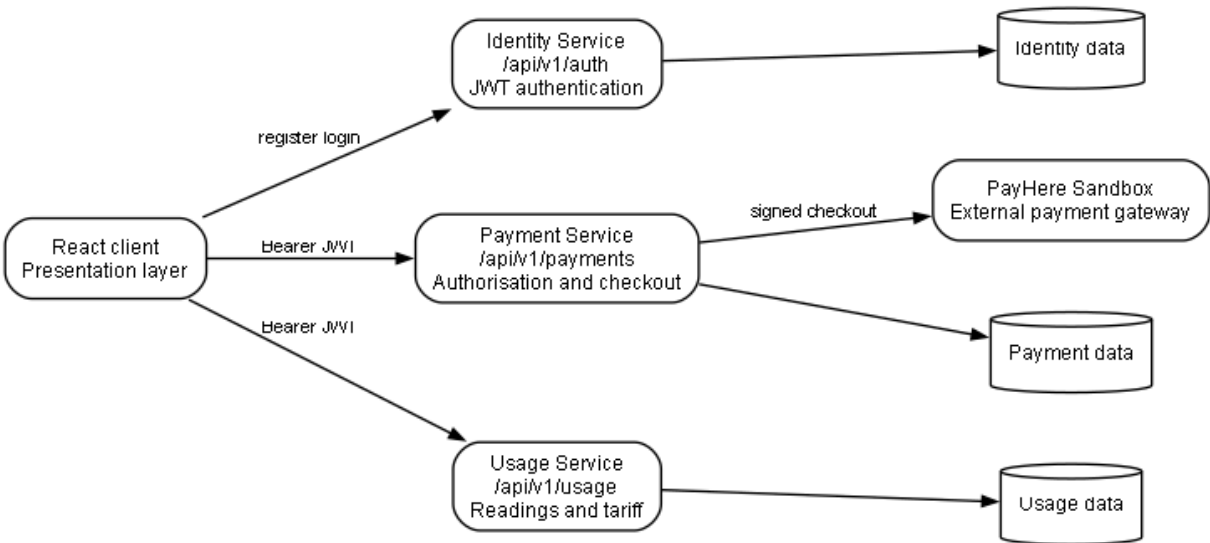


Figure 3 Black and white system architecture

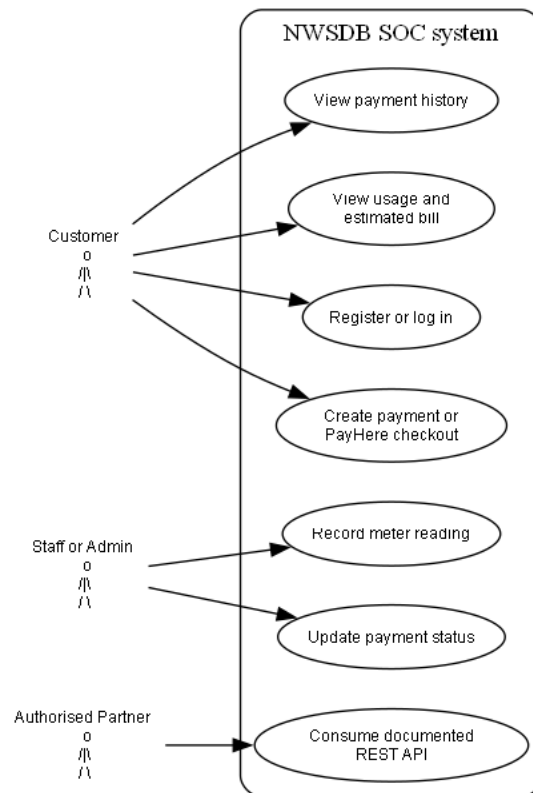


Figure 4 UML use case diagram

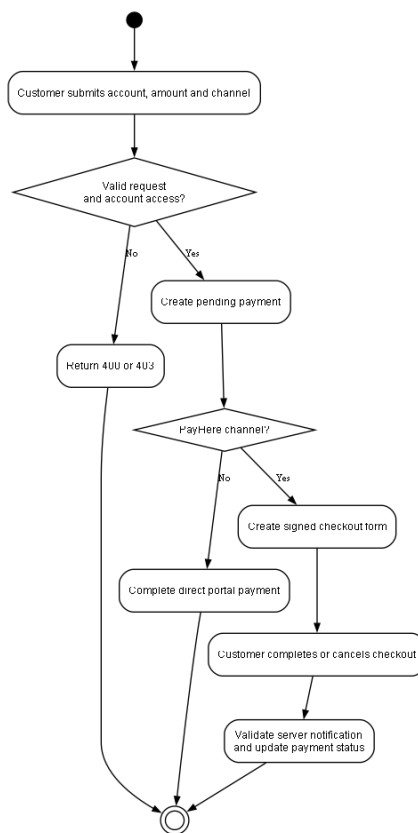


Figure 5 UML activity diagram for payment

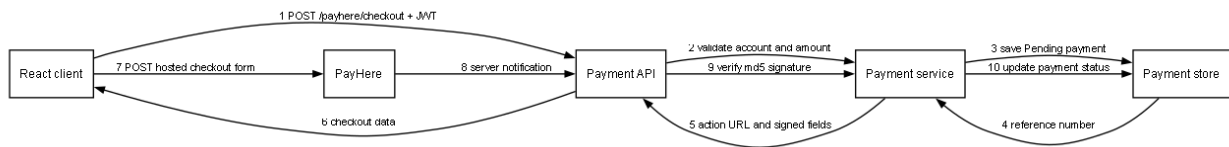


Figure 6 Payment sequence diagram

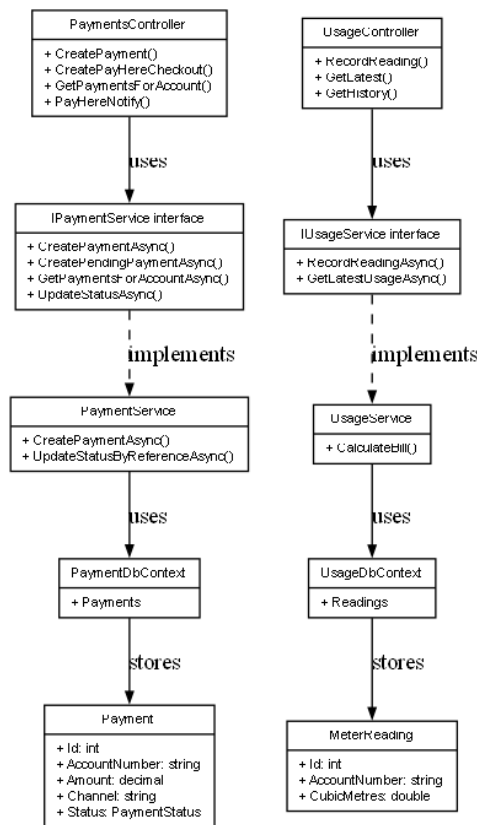


Figure 7 UML class diagram

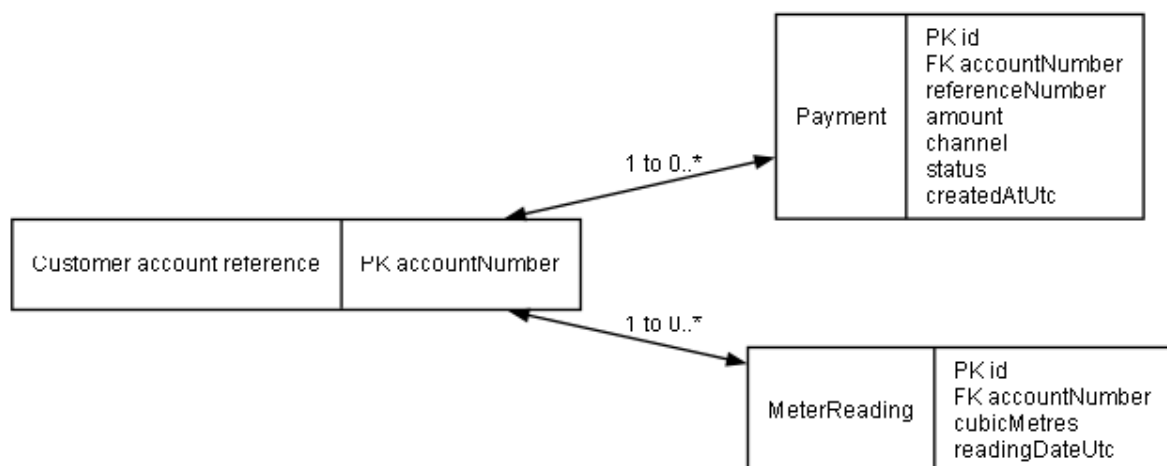


Figure 8 Entity relationship diagram

Task 2 Code Quality Reusability and Maintainability

2.7 Coding Standards and Separation of Concerns

The code is organised by service, controller, model and service-layer responsibility. Interfaces make the main business services replaceable and testable. Validation is performed at controller and service level, so invalid payment amounts and negative or decreasing readings are rejected close to their source. The payment and usage APIs use account claims to protect customer data, while meter recording is restricted to Staff and Admin. Passwords are hashed by the identity service rather than stored as plaintext. These choices meet the rubric expectation for modular, reusable and maintainable code.

2.8 Review Findings and Limitations

The review also identified four limitations that should be acknowledged in the submission. First, EF Core InMemory databases are reset when a service restarts and cannot support independent replicas with a shared record of truth. Second, the services use a development JWT signing-key fallback and AllowAnyOrigin CORS; production must require a secret from configuration and limit origins. Third, Kubernetes manifests show Deployments, Services, probes and resource limits, but the current ingress routes only to the client service; API path routing or a gateway must be added before that manifest is described as a working end-to-end production deployment. Fourth, the service test projects are stronger than identity and browser test coverage. These are improvement areas, not evidence of failure of the implemented academic prototype.

2.9 Evidence Accuracy

A small documentation correction is also important for accuracy. The README introduction refers to two APIs, but the repository contains Identity Service in addition to Payment Service and Usage Service. The report consistently describes three backend services. The solution-level test command did not discover the Usage test project in the observed local verification run, although running each test project directly produced 10 passing Payment tests and 8 passing Usage tests. For submission evidence, use the individual project commands or repair solution-level discovery, and show the actual result instead of claiming an unsupported test count.

Task 3 Testing and Debugging

3.1 Test Strategy and Results

Testing was performed at unit and API-integration level. The current verification run executed 10 Payment Service tests and 8 Usage Service tests, with all 18 passing. Payment tests cover successful payment creation, invalid amounts, account filtering, unknown records, health checks and the PayHere checkout hash. Usage tests cover tariff boundaries, monotonic tariff behaviour, negative readings, consumption calculation and unknown accounts. The use of an isolated EF Core InMemory database gives each unit test a repeatable data set. Payment HTTP tests use WebApplicationFactory and HttpClient to exercise controller routes and response codes.

3.2 Manual System Testing

Manual system testing is still required because an assessor must see that the React client, authentication service and protected APIs operate as one flow. The recommended demonstration is customer registration or login, dashboard loading, usage-history retrieval, an attempted staff-only action returning 403, a successful staff meter reading, a payment record, PayHere sandbox checkout, and service health checks. Screenshots should show both the browser outcome and, where useful, Swagger or terminal evidence. Never include a merchant secret, JWT or password in a screenshot.

3.3 Debugging Record

The documented debugging record includes the HTTPS redirect losing an authorisation header, a request-contract mismatch between paymentMethod and channel, missing landing-page styling and an improved authentication diagnostic path. During the final PayHere verification, an unauthorised payment request was resolved by ensuring the literal merchant secret from the correct website Integration was used in the checksum. This is a useful authentic debugging example because it links observed gateway behaviour to configuration, request hashing and a validated correction.

3.4 White Box and Black Box Coverage

The test strategy combines black-box and white-box thinking. At black-box level, the caller sees an HTTP route, supplied input and resulting status or JSON response. At white-box level, tests deliberately exercise internal decision points: payment amount greater than zero, tariff boundaries, a new reading lower than the previous reading, an unknown

record, and account ownership. The evidence is stronger when the test name, input and expected result can be read directly in the terminal output or test explorer. A failure should be captured before its correction only if it is a real defect encountered during development; fabricated failures or invented coverage must not be used.

3.5 Recommended Additional Tests

Recommended additional tests are an Identity Service integration test for registration and login, a JWT test showing the correct account-number claim, a Usage controller integration test for Staff and Customer roles, a PayHere notification-signature rejection test and browser-level tests for the key React journeys. These additions would move the project closer to the rubric's excellent descriptor of comprehensive unit, integration and functional coverage. They are presented as future work because the current repository does not contain those automated checks.

Screenshot 1

Insert public home page or login page showing the NWSDB branding and entry point.

Screenshot 2

Insert customer dashboard showing current usage and payment history for one account.

Screenshot 3

Insert staff or admin dashboard showing a successful meter-reading operation.

Screenshot 4

Insert Swagger or browser evidence of a 403 Forbidden response for a customer attempting a staff-only action.

Screenshot 5

Insert terminal evidence of the 18 passing automated tests. Do not use an old test-count screenshot.

Screenshot 6

Insert PayHere sandbox checkout or successful return page. Blur or hide personal payment data and never show the merchant secret.

Task 4 Deployment Techniques

4.1 Server and Docker Deployment

A single server deployment is the simplest option for a small demonstration: the APIs and client can run as processes behind a reverse proxy. It is inexpensive and easy to understand, but has limited resilience and makes environment consistency harder to maintain. Docker is a better repeatable-development and packaging option for this project. Each Dockerfile packages one bounded component and docker-compose.yml starts identity, payment, usage and client containers on a shared network. Containers package the code and its dependencies in isolated processes, which improves portability between development, test and deployment environments (Docker, 2026).

4.2 Kubernetes Deployment

Kubernetes becomes appropriate when availability, controlled rollouts and independent service scaling justify its operational cost. The repository includes a namespace, Deployments, ClusterIP Services, resource requests and liveness/readiness probes. A Kubernetes Deployment maintains the declared pod state and supports controlled rollout (Kubernetes, 2026a), while a Service provides a stable network endpoint for changing pods (Kubernetes, 2026b). The current choice of one replica for stateful InMemory services is technically correct for the prototype. With PostgreSQL or another managed durable store, payment and usage replicas could scale horizontally according to demand.

4.3 Recommended Deployment Path

The recommended progression is Docker Compose for local demonstration, a managed container service or small server for early release, and Kubernetes only after persistent storage, secret management, API ingress rules, TLS and monitoring have been completed. PayHere callback URLs must be public HTTPS URLs in a deployed environment. This staged approach balances cost, maintainability and scalability rather than adopting Kubernetes without the supporting operational design.

4.4 Production Readiness

A production deployment should use environment-specific configuration. Merchant IDs, PayHere secrets, JWT keys and database connection strings must be held in a managed secret store or Kubernetes Secret and injected at runtime. Log output should contain request identifiers and operational events but never passwords, bearer tokens or payment secrets. Database migrations and backups should be part of the release process, and readiness checks should verify that each service can reach its critical dependencies. An ingress controller or API gateway should route /api/v1/auth, /api/v1/payments and /api/v1/usage to the correct ClusterIP service while the client remains the public website entry point.

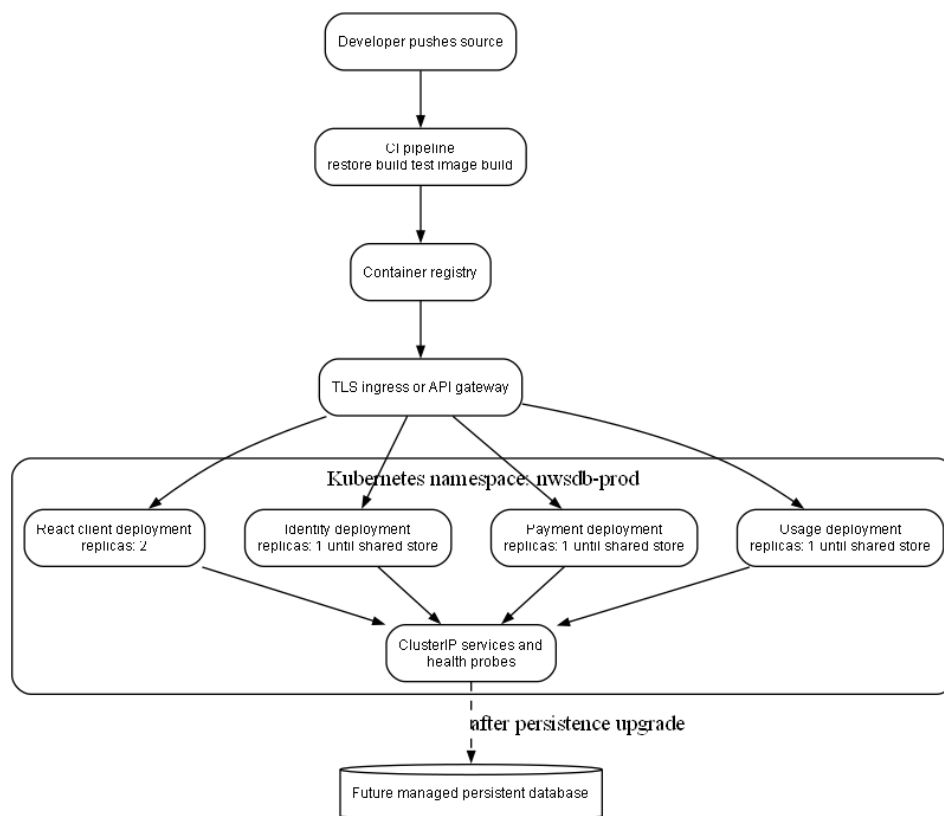


Figure 9 Deployment architecture

Screenshot 7

Insert Docker Compose containers or Kubernetes pod and service status if you actually run them.

Overall Conclusion

The NWSDB solution satisfies the central SOC objective by exposing distinct identity, usage and payment capabilities as protected REST services consumed by a client application. The architecture is better suited to NWSDB than a monolith because future partners can integrate through stable contracts and individual capabilities can evolve separately. The application demonstrates usable customer and operational flows, testing, Docker packaging, Kubernetes design material and an external payment-gateway integration.

For the strongest assessment submission, the report should be accompanied by the requested screenshots and should present the production limitations transparently. The next technical priorities are persistent storage, production secrets and CORS policy, API routing in Kubernetes, additional Identity Service and browser tests, and deployment evidence. These improvements naturally extend the current service boundaries without invalidating the SOC design.

Overall, the assessment criteria are addressed in a connected way rather than as separate features. The comparison establishes why a service-oriented approach is justified; the diagrams and code show how it is implemented; the tests and debugging record show how its behaviour was checked; and the Docker and Kubernetes material explains how it can be deployed. The final screenshots are essential because they turn the written analysis into verifiable evidence of the student's own working system. Once personal details and genuine evidence are inserted, the document can be exported as PDF with the required student ID, module code and assessment identifier in the filename.

References

Docker (2026) What is a container. Available at:

<https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/> (Accessed: 22 September 2026).

Erl, T. (2005) Service Oriented Architecture Concepts Technology and Design. Upper Saddle River: Prentice Hall.

Fielding, R.T. (2000) Architectural Styles and the Design of Network based Software Architectures. Doctoral dissertation. University of California Irvine.

Kubernetes (2026a) Deployments. Available at: <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/> (Accessed: 22 September 2026).

Kubernetes (2026b) Service. Available at: <https://kubernetes.io/docs/concepts/services-networking/service/> (Accessed: 22 September 2026).

Microsoft (2026) Create web APIs with ASP NET Core. Available at:

<https://learn.microsoft.com/en-us/aspnet/core/web-api/> (Accessed: 22 September 2026).

Newman, S. (2021) Building Microservices. 2nd edn. Sebastopol: O Reilly Media.

PayHere (2026) Checkout API. Available at: <https://support.payhere.lk/api-%26-mobile-sdk/checkout-api> (Accessed: 22 September 2026).

Appendix A Screenshot Evidence Checklist

Replace each placeholder in Section 6 and Section 7 with your own dated, readable evidence before you export the final PDF. Keep the caption below each screenshot and remove placeholder text once the image is inserted.

Evidence	What to show	Status
Screenshot 1	Insert public home page or login page showing the NWSDB branding and entry point.	[Insert]
Screenshot 2	Insert customer dashboard showing current usage and payment history for one account.	[Insert]
Screenshot 3	Insert staff or admin dashboard showing a successful meter-reading operation.	[Insert]
Screenshot 4	Insert Swagger or browser evidence of a 403 Forbidden response for a customer attempting a staff-only action.	[Insert]
Screenshot 5	Insert terminal evidence of the 18 passing automated tests. Do not use an old test-count screenshot.	[Insert]
Screenshot 6	Insert PayHere sandbox checkout or successful return page. Blur or hide personal payment data and never show the merchant secret.	[Insert]
Screenshot 7	Insert Docker Compose containers or Kubernetes pod and service status if you actually run them.	[Insert]